משימה 4 — הבקר השבור של פלנקטון

נושאים: דיבוג קוד C קיים, מחרוזות, מערכים, מצביעים, זיכרון דינמי, עבודה עם GDB ובדיקות תקינות זיכרון.

הרצת `./operation_b7/` מהתיקייה הראשית יוצרת את `mission4/data/mission4_controller.b7`. זהו קובץ ראיות אישי; אין לערוך אותו.

תדריך מודיעיני

המצב בבאר שבע חמור. מההתכתבויות ששחזרתם התברר שהבארשבעינייטור, נשק שפותח במימון גורמים איראניים, מסוגל לשבש מערכות תנועה, כניסה ותשתיות עירוניות. דופנשמירץ מתכנן להפעיל אותו בחצות, להשתלט על באר שבע ולשנות את שמה לדופנשמירץ־שבע. הזמן הולך ואוזל.

אבל במהלך שחזור ההתכתבויות גיליתם גם את מה שעשוי להיות הסיכוי היחיד שלנו לסכל את המתקפה: **מנגנון ההשמדה העצמית שדופנשמירץ התקין בבארשבעינייטור**. הוא אפילו חיבר אותו לשליטה מרחוק. אם נצליח להגיע למנגנון ולהפעיל אותו לפני חצות, נוכל להציל את העיר.

בעקבות הגילוי, צוות המודיעין איתר בשרת הגיבוי של הארגון תיקיית תחזוקה שפלנקטון העלה בעקבות תקלות חוזרות בבקר החירום. בתיקייה נמצאו קוד המקור של תוכנת הבקרה ונתוני בדיקה, אך התברר שהתוכנה אינה יציבה ולא ניתן להסתמך עליה להפעלת מנגנון ההשמדה.

התכתבות שנמצאה לצד הקבצים מסבירה היטב את מצב התוכנה:

> **ד״ר דופנשמירץ:** פלנקטון! למה הבקר שלי מתרסק כל הזמן?!
>
> **פלנקטון:** זו מערכת אבטחה. ככה האקרים לא יכולים להשתמש בו.
>
> **ד״ר דופנשמירץ:** וגם אני לא יכול!
>
> **פלנקטון:** אמרתי שהיא מאובטחת.

הקוד שקיבלתם הוא העותק שפלנקטון כתב. הוא מתקמפל, וחלק מהפעולות שלו אפילו נראות תקינות, אבל במצבים אחרים הוא נכשל. במצבו הנוכחי אי אפשר להסתמך עליו כדי לפענח את הוראות הבקרה שמצאנו.

המשימה שלכם

האחריות להחזיר את בקר החירום לפעולה מוטלת עליכם. מצאנו דרך אפשרית להשמיד את הבארשבעינייטור, אך אין לנו תוכנת בקרה שאפשר לסמוך עליה כדי לבצע זאת. עליכם לאתר את התקלות בקוד שפלנקטון השאיר, להבין מה גורם להן ולתקן אותן בלי לפגוע בפעולות התקינות.

הצלחתכם תאפשר לצוות החקירה להתקדם אל מנגנון ההשמדה העצמית. אבל שימו לב: **במשימה זו תשחזרו את ממשק החירום בלבד. הפעלת המנגנון עצמו עדיין אינה אפשרית ותידרש בהמשך המבצע.**

הפעם לא תכתבו פונקציות מאפס. המימושים כבר קיימים ב־`controller.c`, ובכל אחד מארבעת השלבים עליכם:

1. להריץ את בדיקת השלב ולזהות התנהגות שאינה תואמת את הדרישות.
2. לקרוא את המימוש, לחקור את הסיבה לתקלה ולתקן אותו בלי לשנות את חתימת הפונקציה או את החוזה שלה.
3. להריץ שוב את הבדיקה ולוודא שהתיקון עובד גם במקרה הגבול הרלוונטי.
4. להסביר בקצרה בדוח מה הייתה התקלה, מדוע התיקון עובד ואיזה מקרה בדיקה מאמת זאת.

בנוסף, **בשלב אחד בלבד לבחירתכם מבין A–D** תבצעו חקירה קצרה באמצעות **GDB** ותתעדו תצפית אחת מתוך הריצה. אין צורך להשתמש ב־GDB בכל ארבעת השלבים. אפשר להיעזר בו, ב־sanitizers או בבדיקות נוספות גם בשלבים האחרים. בהמשך מופיע תרגול מודרך למי שלא השתמש בכלי בעבר.

אפשר לבדוק כל שלב בנפרד, גם אם שלבים אחרים טרם תוקנו. כל שלב תקין מאפשר לחוקרים לפענח עוד חלק מהראיות. רק לאחר שכל ארבעת השלבים יעברו בהצלחה תוכלו להפיק את קוד ההשלמה של המשימה.

מותר לערוך: `mission4/controller.c` בלבד. אין לשנות את קובצי התשתית שב־`../support/mission4_`, מבני הנתונים, הספרייה המקומפלת או קובצי הראיות. אין להחליף את עיבוד הנתונים בפלט קבוע.

העבודה מתבצעת על **עותק מקומי ומבודד** ועל נתוני בדיקה. אינכם מתחברים למכשיר אמיתי או למערכת עירונית, ואינכם שולחים פקודות ברשת. במשימה זו תכינו את הדרך להפעלת מנגנון ההשמדה; ההפעלה עצמה תידרש בהמשך החקירה.

מבני הנתונים

אלה המבנים שמוגדרים ב־`controller.h`. אין צורך לשנות אותם.

```
typedef struct {
    int id;
    char name[32];
    char role[48];
} ControlUnit;
typedef struct {
    int id;
    char payload[32];
} ControlCommand;
typedef struct {
    int id;
    char *text;
} PendingCommand;
```

הרצת הבדיקות

מתוך `/mission4`:

```
make
make test-a
make test-b
make test-c
make test-d
```

כל פקודה בודקת שלב אחד בלבד. לאחר שתיקנתם את כל ארבעת השלבים, הריצו:

```
make run
```

רק הריצה המלאה מאמתת שהבקר תקין ומציגה קוד השלמה אישי בפורמט `B7-M4-XXXXXXXX`.

שלב A — קליטת פקודות

הבקר מקבל פקודות כטקסט. הפונקציה הבאה אמורה להעביר פקודה למאגר שהוקצה עבורה, בלי לשנות את הפקודה המקורית.

```
int controller_parse_command(const char *input, char *output,
                             size_t capacity);
```

התנהגות נדרשת:

- `input` היא מחרוזת C תקינה המסתיימת ב־`'\0'`. `output` מצביע למאגר שניתן לכתוב אליו, בגודל `capacity` בתים.
- אם יש מקום לכל הפקודה ולתו הסיום, יש להעתיק אותה במלואה ולהחזיר `1`. גם מחרוזת ריקה היא פקודה חוקית.
- אם אחד המצביעים הוא `NULL`, הקיבולת היא אפס או שאין מספיק מקום, יש להחזיר `0`. אין להעתיק פקודה חלקית.
- בכישלון, כאשר `output` ניתן לכתיבה ו־`capacity > 0`, עליו להכיל מחרוזת ריקה תקינה.
- המקור והיעד אינם חופפים. אין לשנות את המקור או לגשת לזיכרון מחוץ למאגרים.

התחילו ב־`make test-a` וקראו את הודעת הכישלון. בדקו באילו תנאים הפקודה אמורה להתקבל ובאילו תנאים עליה להידחות. אם בחרתם בשלב A לחקירת GDB, אפשר להתחיל בנקודת עצירה בכניסה לפונקציה, לבדוק את הקלט ואת הקיבולת ולהתקדם שורה אחר שורה. אין צורך לדעת מראש איזה ערך גרם לתקלה. לאחר התיקון הריצו שוב `make test-a`; ניתן להיעזר גם ב־`make debug-a`.

שלב B — איתור רכיבי בקרה

המידע שנמצא בחבילת התחזוקה מתייחס לרכיבים שונים של המכשיר. כדי לבדוק אותם, צריך להיות מסוגלים למצוא רכיב במערך.

```
ControlUnit *controller_find_unit(ControlUnit *units, size_t count,
                                  int index);
```

התנהגות נדרשת:

- אינדקס חוקי הוא מספר בין `0` ל־`count - 1`. במקרה כזה יש להחזיר מצביע **לאיבר המקורי** במערך.
- עבור אינדקס שלילי, אינדקס השווה ל־`count` או גדול ממנו, מערך `NULL` או מערך ריק — יש להחזיר `NULL`.
- אין לשנות את המערך, להקצות זיכרון, להחזיר עותק של המבנה או לחשב כתובת שאינה בתחום המערך החוקי.

הריצו `make test-b`. השוו בין תחום האינדקסים החוקיים לבין ההתנהגות בפועל, ותקנו את הסיבה לחריגה מהחוזה. לאחר מכן הריצו שוב את הבדיקה. אפשר להיעזר ב־`make gdb-b` או ב־`make debug-b` לפי הצורך.

שלב C — ניהול תור הפקודות

הבקר מחזיק תור של פקודות. כאשר פקודה אחת מוסרת, שאר הפקודות צריכות להישאר בתור, באותו סדר.

```
int controller_remove_command(ControlCommand *commands, size_t *count,
                              size_t capacity, size_t index);
```

התנהגות נדרשת:

- להסיר את הפקודה ב־`index`, להזיז את הפקודות שאחריה מקום אחד שמאלה, ולשמור על הסדר שלהן.
- להקטין את `count*` רק לאחר הסרה מוצלחת.
- לאפס את התא שהתפנה בסוף החלק הפעיל: `id == 0` ו־`payload[0] == '\0'`.
- לתמוך בהסרת הפקודה הראשונה, האחרונה, האמצעית או היחידה בתור.
- להחזיר `0` בלי לשנות את התור או את המונה אם יש מצביע `NULL`, התור ריק, `index >= *count` או `count* > capacity`.
- להחזיר `1` בהצלחה, בלי גישה מחוץ למערך או גלישה בחישובי גודל.

הריצו `make test-c`. השוו בין תור הפקודות לפני ההסרה ולאחריה: האם הפקודה הנכונה הוסרה, והאם יתר הפקודות נשארו שלמות ובסדר הנכון? לאחר התיקון הריצו שוב את הבדיקה. `make gdb-c` ו־`make debug-c` זמינים ככלי עזר.

שלב D — תזמון וביצוע פקודות

לא כל פקודה מתבצעת מיד. חלק מהן נשמרות עד שמגיע הזמן לעבד אותן. לכן הבקר צריך לדעת לשמור בקשה, להשאיר אותה זמינה ולאחר מכן לטפל בה בלי לאבד מידע.

```
int controller_schedule(PendingCommand **pending, int id,
                        const char *text);
int controller_execute_pending(PendingCommand **pending, char *output,
                               size_t capacity);
```

`controller_schedule` — שמירת פקודה:

- `pending` חייב להצביע לתא שמכיל בתחילה `NULL`; `id` חייב להיות חיובי ו־`text` אינה `NULL`.
- בהצלחה יש להקצות `PendingCommand` חדש ומחרוזת עצמאית המכילה עותק מלא של `text`, להציב את המצביע בתא ולהחזיר `1`.
- הפקודה צריכה להישאר תקינה גם אם מחרוזת המקור משתנה או חדלה להתקיים.
- אם התא תפוס, יש לדחות פקודה חדשה בלי לשנות את הקיימת.

- אם הקצאה נכשלת, יש לשחרר את מה שהוקצה באותו ניסיון, לשמור על המצב הקודם ולהחזיר `0`.

`controller_execute_pending` — עיבוד הפקודה:

- יש לוודא שקיימת פקודה תקינה ושב־`output` יש מקום לטקסט המלא ול־`'\0'`.
- בהצלחה יש להעתיק את הטקסט, לשחרר בדיוק פעם אחת את הזיכרון שבבעלות הפקודה, להציב `pending* = NULL` ולהחזיר `1`.
- אם האימות נכשל או המאגר קטן מדי, יש להחזיר `0` ולהשאיר את הפקודה זמינה לניסיון חוזר.
- בכישלון, אם `output` ניתן לכתיבה ו־`capacity > 0`, עליו להכיל מחרוזת ריקה. אם אין פקודה ממתינה, יש להיכשל בבטחה.

הריצו `make test-d`. בדקו מה אמור להישמר בזיכרון אחרי תזמון הפקודה, ומתי מותר לשחרר אותו. אם בדיקות הזיכרון מצביעות על בעיה, אפשר להשתמש ב־`make debug-d` כדי לקבל מידע נוסף; GDB זמין באמצעות `make gdb-d`. לאחר התיקון ודאו גם שאפשר לעבד את הפקודה בהצלחה ולנקות את הזיכרון בבטחה.

היכרות קצרה עם GDB

GDB הוא כלי שמאפשר לעצור תוכנית בזמן ריצה, לבדוק את ערכי המשתנים ולהתקדם בקוד שורה אחר שורה. אין צורך בניסיון קודם כדי לבצע את המשימה הזאת: נלמד רק את הפעולות הבסיסיות, ונדרוש תיעוד GDB **בשלב אחד בלבד לבחירתכם**.

תרגול מודרך: לאן נעלם התקציב?

לפני שמתקנים את הבקר, מתנסים על דוגמה קצרה ונפרדת. הקובץ `debug/gdb_demo.c` מחשב את היתרה מתקציב של 100 אחרי הוצאה של 25. התשובה הצפויה היא 75, אבל התוכנית מחזירה 125. ננסה לגלות מדוע, בלי להניח מראש איפה השגיאה.

מתוך `/mission4` הריצו:

```
make gdb-demo
gdb ./gdb_demo
```

כעת אתם בתוך GDB. הסימון `(gdb)` הוא סימן הפקודה של הכלי ואין צורך להקליד אותו. תחילה עוצרים בכניסה לפונקציה שמחשבת את היתרה, ואז מפעילים את התוכנית:

```
(gdb) break remaining_budget
(gdb) run
```

כשהתוכנית נעצרת, בדקו מה היא קיבלה. `info args` מציגה את הפרמטרים; `list` מציגה את שורות הקוד באזור שבו נעצרנו:

```
(gdb) info args
(gdb) list
```

```
(gdb) print budget
(gdb) print spent
```

כעת התקדמו באמצעות `next`, שורה בכל פעם, עד **אחרי** השורה שמחשבת את המשתנה `remaining`. בדקו את הערך:

```
(gdb) next
(gdb) print remaining
```

אם עדיין לא הגעתם לשורת החישוב, השתמשו שוב ב־`next` ורק אז בדקו את `remaining`. אל תסתמכו על ערך של משתנה לפני שאותחל. לאחר ביצוע החישוב אתם אמורים לראות 125 במקום 75. חזרו לשורת החישוב באמצעות `list` ונסו להסביר כיצד התקבלה התוצאה הלא נכונה. אפשר להמשיך ולצאת:

```
(gdb) continue
(gdb) quit
```

זו דרך העבודה שבה נשתמש גם בבקר: מתחילים מתוצאה לא נכונה, עוצרים בפונקציה הרלוונטית, בודקים מה התקבל, מתקדמים עד לשורה החשודה ומסבירים מה ראינו. אין צורך לנחש מראש את הקלט המדויק או את מספר השורה של הבאג.

פקודה	מה היא עושה?
`break FUNCTION`	מציבה נקודת עצירה בכניסה לפונקציה
`run`	מתחילה את הרצת התוכנית
`next`	מתקדמת לשורה הבאה בלי להיכנס לפונקציות שנקראות ממנה
`print expression`	מציגה את ערכו של משתנה או ביטוי
`continue`	ממשיכה עד נקודת העצירה הבאה או סוף הריצה
`list`	מציגה שורות קוד סביב מיקום העצירה
`info args` / `info locals`	מציגות פרמטרים ומשתנים מקומיים
`step`	נכנסת לפונקציה שנקראת בשורה הנוכחית, אם אפשר
`backtrace`	מציגה את שרשרת הקריאות; שימושית במיוחד אחרי קריסה
`quit`	יוצאת מ־GDB

GDB בשלבי המשימה

בחרו שלב אחד מבין A–D לחקירת GDB. המטרה היא להבין תצפית ממשית מתוך ההרצה, ולא להשתמש בכמה שיותר פקודות. למשל, אפשר להציג ערך פרמטר, מצב של מבנה או מסלול ביצוע, ולהסביר כיצד הם קשורים לתקלה.

מתוך `/mission4`, הפקודה של השלב שבחרתם מכינה קובץ הרצה עם מידע לדיבוג (`g -O0-`) ופותחת אותו ב־GDB:

```
make gdb-a
make gdb-b
make gdb-c
make gdb-d
```

יש להריץ **רק את הפקודה של השלב שבחרתם**, ולא את כולן. תוכלו להתחיל ב־`break` על שם הפונקציה הנבדקת, להריץ `run`, לבדוק את הפרמטרים ולהתקדם עם `next`. ייתכן שהפונקציה תיקרא יותר מפעם אחת במהלך הבדיקה; במידת הצורך השתמשו ב־`continue` כדי לעבור לקריאה הבאה.

אין חובה להשתמש בנקודות עצירה מותנות, ב־`backtrace`, בכתובות זיכרון מספריות או בפקודות מתקדמות. אין צורך לצרף צילומי מסך או תמליל מלא של הריצה.

בדיקות זיכרון וכלי עזר נוספים

לצד GDB עומדים לרשותכם גם AddressSanitizer ו־UndefinedBehaviorSanitizer. אפשר להריץ את הבדיקה הרלוונטית לשלב שבו אתם עובדים:

```
make debug-a
make debug-b
make debug-c
make debug-d
```

כלים אלה עשויים לזהות גישה מחוץ לגבולות זיכרון או שימוש בזיכרון שאינו תקף. במיוחד בשלב D, הודעת sanitizer יכולה לעזור לאתר בעיית זיכרון. תוכלו להיעזר גם בקריאת הקוד, בבדיקות השלב וב־GDB, לפי הצורך. **השימוש ב־GDB לצורך התיעוד נדרש רק בשלב אחד; בשאר השלבים שיטת החקירה נתונה לבחירתכם.**

לפני ההגשה

לאחר שכל ארבעת השלבים עברו בהצלחה, הריצו `make run` ורשמו את קוד ההשלמה ואת הממצאים שהתקבלו. **המשימה מסתיימת בשחזור ממשק החירום, לא בהפעלת מנגנון ההשמדה.**

מלאו את החלק של משימה 4 ב־`investigation_report.md`, בהתאם לדרישות הבאות:

1. ממצאי החקירה

תארו בקצרה מה הצלחתם לשחזר בבקר ומה עדיין מונע את השלמת הפעולה. ציינו את קוד ההשלמה האישי.

2. תיעוד התיקונים — שלבים A–D

לכל אחד מארבעת השלבים הסבירו בקצרה:

- מה היה שורש התקלה בקוד המקורי?
- מה שיניתם ומדוע התיקון עובד?
- איזו בדיקה מאמתת את התיקון? ציינו קלט או תרחיש ואת התוצאה הצפויה.

מספיקות כמה שורות לכל שלב. אין צורך לתאר מחדש את כל הקוד.

3. חקירת GDB אחת

בחרו שלב אחד בלבד מבין A–D ותעדו היכן עצרתם, באילו פקודות עיקריות השתמשתם, מה ראיתם בפועל וכיצד התצפית סייעה להבין את התקלה.

אין צורך בצילומי מסך או בתמליל מלא של הריצה.

היקף הדוח: תשובות קצרות, מדויקות ומנומקות מספיקות. אין דרישת מינימום לאורך.

חלוקת הניקוד במשימה: 50% קוד, 25% הסבר התקלות והתיקונים, 15% בדיקות ואימות, 10% חקירת GDB אחת.

מגישים: את `mission4/controller.c` ואת `investigation_report.md` המעודכן. אין להגיש קובצי הרצה, ספריות מקומפלות או קובצי ראיות.